

JAVA PROGRAMMING – CO5

NAME : U SUBIKSHA

REG NO : 192424135

1. Introduction

The Employee Management and Payroll Application is a desktop-based application proposed for an organization that currently maintains employee information using spreadsheets and manually prepared records. As the number of employees increases, maintaining accurate employee details, updating salary information, retrieving employee information and preparing payroll-related information becomes difficult. The proposed system uses Java AWT for the graphical user interface and MySQL for persistent data storage, with JDBC used for database connectivity.

2. Objectives

Develop a desktop-based Employee Management Application using Java AWT.

Store employee information persistently using MySQL.

Connect the Java application to MySQL using JDBC.

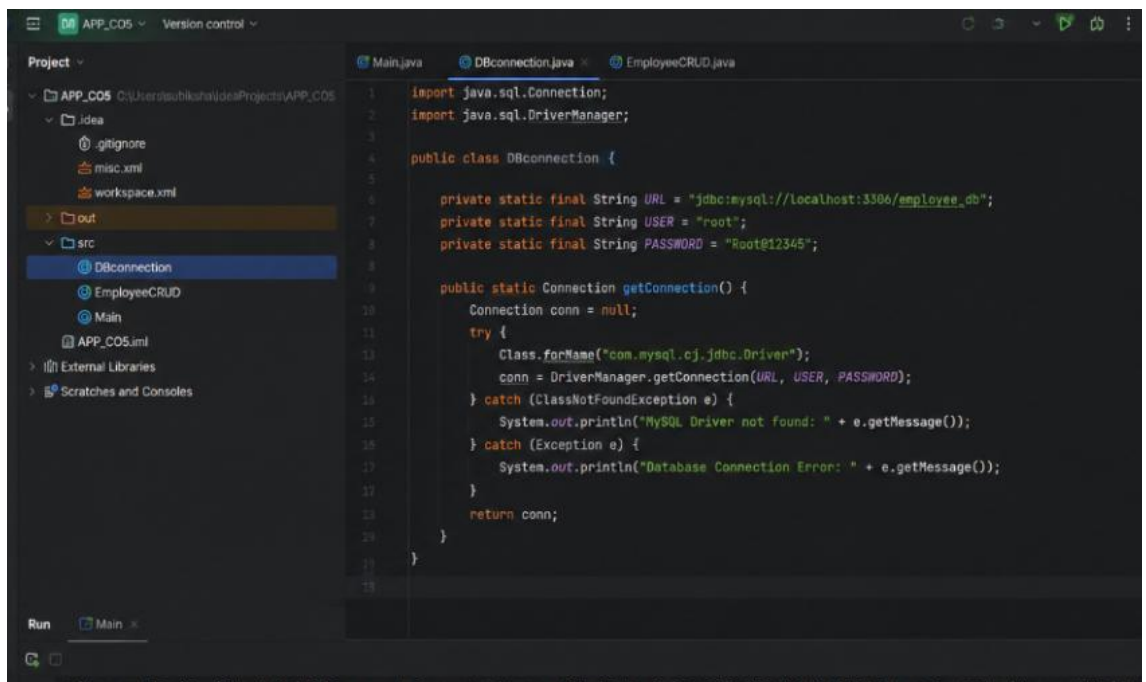
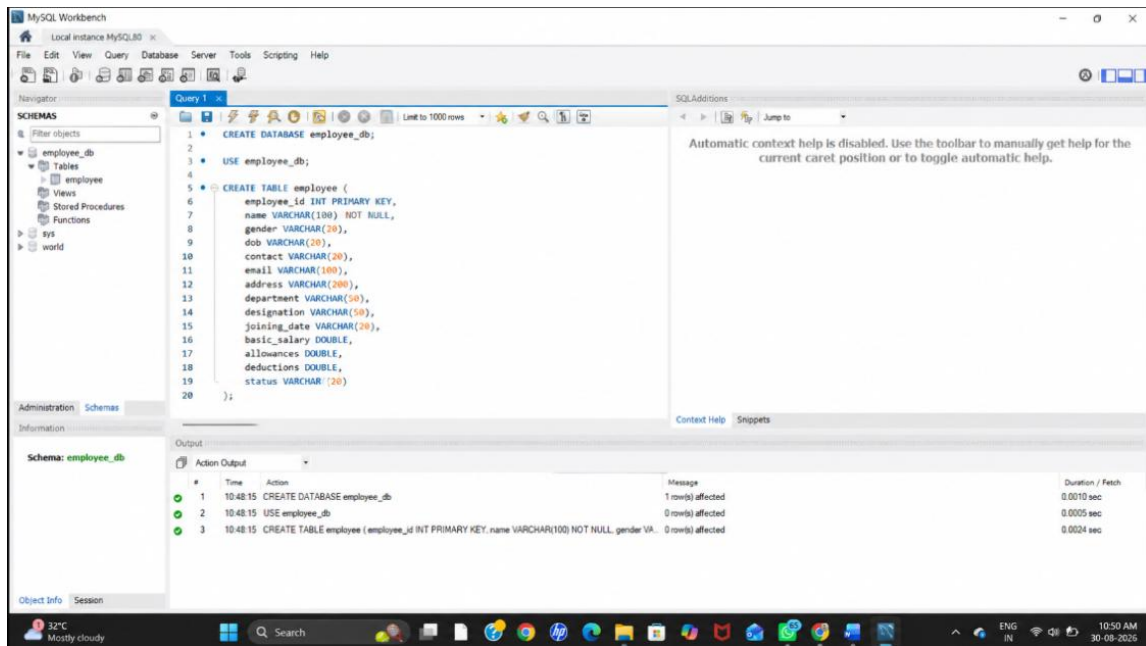
Implement employee registration, retrieval, search, update and delete operations.

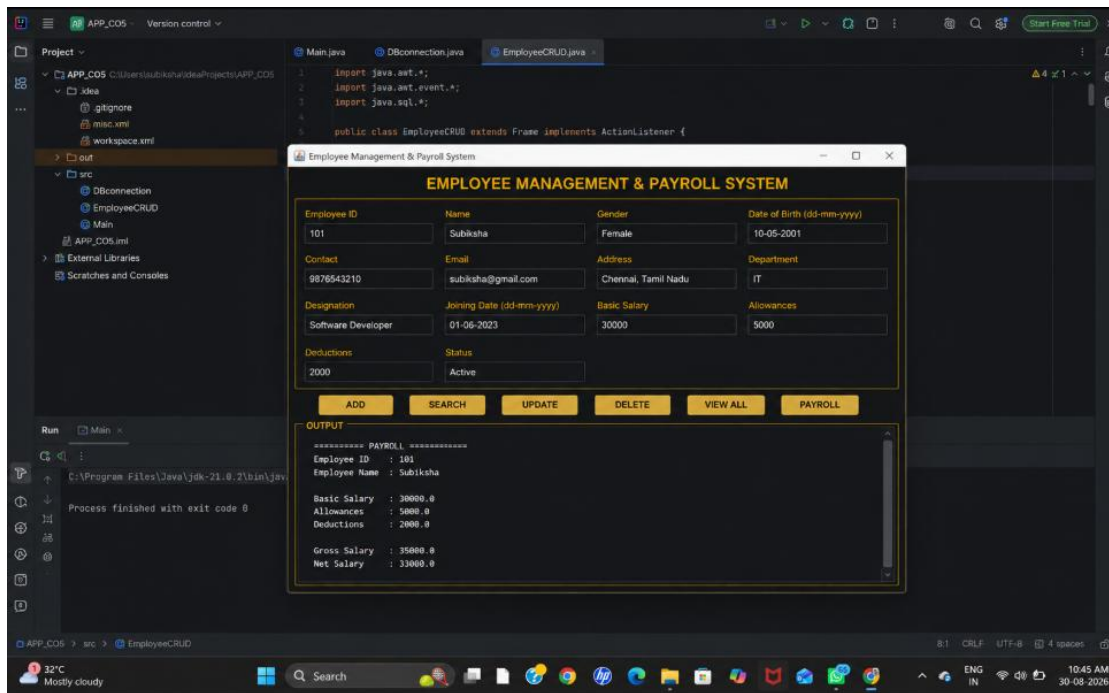
Provide payroll calculation for gross salary and net salary.

Validate mandatory fields, salary values and Employee ID uniqueness.

Handle database and application-level errors using exception handling.

Provide a modular design that can be extended with future HR functions





OUTPUT :



CODE

```
import java.awt.*;
import java.awt.event.*;
import java.sql.*;

public class EmployeeCRUD extends Frame implements ActionListener {
    TextField txtId, txtName, txtDob, txtContact, txtEmail;
    TextField txtAddress, txtJoiningDate, txtBasic, txtAllowances, txtDeductions;
    Choice gender, department, designation, status;
    Button btnAdd, btnSearch, btnUpdate, btnDelete, btnView, btnPayroll,
    btnClear;
    TextArea txtArea;

    Color dark = new Color(18, 18, 18);
    Color darkPanel = new Color(28, 28, 28);
    Color darkField = new Color(40, 40, 40);
    Color gold = new Color(212, 175, 55);
    Color lightGold = new Color(245, 215, 130);
    Color white = new Color(235, 235, 235);

    public EmployeeCRUD() {
        setTitle("Employee Management - CRUD Application");
        setSize(950, 650);
        setLayout(new BorderLayout(10, 10));
        setBackground(dark);
```

```
Label lblTitle = new Label(
    "◆ EMPLOYEE MANAGEMENT SYSTEM ◆",
    Label.CENTER);
lblTitle.setFont(new Font("Serif", Font.BOLD, 22));
lblTitle.setForeground(lightGold);
lblTitle.setBackground(dark);
add(lblTitle, BorderLayout.NORTH);
Panel formPanel = new Panel(new GridLayout(7, 4, 8, 8));
formPanel.setBackground(darkPanel);
txtId = createTextField();
txtName = createTextField();
txtDob = createTextField();
txtContact = createTextField();
txtEmail = createTextField();
txtAddress = createTextField();
txtJoiningDate = createTextField();
txtBasic = createTextField();
txtAllowances = createTextField();
txtDeductions = createTextField();
gender = new Choice();
gender.add("Male");
gender.add("Female");
gender.add("Other");
styleChoice(gender);
```

```
department = new Choice();
department.add("HR");
department.add("IT");
department.add("Finance");
department.add("Marketing");
department.add("Sales");
styleChoice(department);
designation = new Choice();
designation.add("Manager");
designation.add("Developer");
designation.add("Analyst");
designation.add("HR Executive");
designation.add("Accountant");
styleChoice(designation);
```

```
status = new Choice();
status.add("Active");
status.add("Inactive");
styleChoice(status);
```

```
addField(formPanel, "Employee ID", txtId);
addField(formPanel, "Name", txtName);
addChoice(formPanel, "Gender", gender);
addField(formPanel, "Date of Birth", txtDob);

addField(formPanel, "Contact Number", txtContact);
```

```
addField(formPanel, "Email", txtEmail);
addField(formPanel, "Address", txtAddress);
addChoice(formPanel, "Department", department);

addChoice(formPanel, "Designation", designation);
addField(formPanel, "Joining Date", txtJoiningDate);
addField(formPanel, "Basic Salary", txtBasic);
addField(formPanel, "Allowances", txtAllowances);

addField(formPanel, "Deductions", txtDeductions);
addChoice(formPanel, "Status", status);

add(formPanel, BorderLayout.CENTER);

txtArea = new TextArea();
txtArea.setBackground(Color.BLACK);
txtArea.setForeground(white);
txtArea.setFont(new Font("Monospaced", Font.PLAIN, 14));

add(txtArea, BorderLayout.SOUTH);

Panel buttonPanel = new Panel(
    new FlowLayout(FlowLayout.CENTER, 8, 8));

buttonPanel.setBackground(dark);
```

```
btnAdd = createButton("ADD");  
btnSearch = createButton("SEARCH");  
btnUpdate = createButton("UPDATE");  
btnDelete = createButton("DELETE");  
btnView = createButton("VIEW ALL");  
btnPayroll = createButton("PAYROLL");  
btnClear = createButton("CLEAR");
```

```
buttonPanel.add(btnAdd);  
buttonPanel.add(btnSearch);  
buttonPanel.add(btnUpdate);  
buttonPanel.add(btnDelete);  
buttonPanel.add(btnView);  
buttonPanel.add(btnPayroll);  
buttonPanel.add(btnClear);
```

```
add(buttonPanel, BorderLayout.SOUTH);
```

```
btnAdd.addActionListener(this);  
btnSearch.addActionListener(this);  
btnUpdate.addActionListener(this);  
btnDelete.addActionListener(this);  
btnView.addActionListener(this);  
btnPayroll.addActionListener(this);  
btnClear.addActionListener(this);
```



```
addWindowListener(new WindowAdapter() {  
    public void windowClosing(WindowEvent e) {  
        dispose();  
        System.exit(0);  
    }  
});
```

```
    setLocationRelativeTo(null);  
    setVisible(true);  
}
```

```
TextField createTextField() {  
  
    TextField text = new TextField();  
  
    text.setBackground(darkField);  
    text.setForeground(white);  
  
    return text;  
}
```

```
void styleChoice(Choice choice) {  
  
    choice.setBackground(darkField);  
    choice.setForeground(white);  
}
```

```
void addField(Panel panel, String name, TextField field) {
```

```
    Label label = new Label(name);
```

```
    label.setForeground(lightGold);
```

```
    label.setBackground(darkPanel);
```

```
    panel.add(label);
```

```
    panel.add(field);
```

```
}
```

```
void addChoice(Panel panel, String name, Choice choice) {
```

```
    Label label = new Label(name);
```

```
    label.setForeground(lightGold);
```

```
    label.setBackground(darkPanel);
```

```
    panel.add(label);
```

```
    panel.add(choice);
```

```
}
```

```
Button createButton(String name) {
```

```
    Button button = new Button(name);
```

```
button.setBackground(gold);
button.setForeground(Color.BLACK);
button.setFont(new Font("Arial", Font.BOLD, 12));

return button;
}

public void actionPerformed(ActionEvent e) {

    if (e.getSource() == btnAdd) {
        addEmployee();
    }

    else if (e.getSource() == btnSearch) {
        searchEmployee();
    }

    else if (e.getSource() == btnUpdate) {
        updateEmployee();
    }

    else if (e.getSource() == btnDelete) {
        deleteEmployee();
    }
}
```

```

else if (e.getSource() == btnView) {
    viewEmployees();
}

else if (e.getSource() == btnPayroll) {
    calculatePayroll();
}

else if (e.getSource() == btnClear) {
    clearFields();
}
}

boolean validateData() {

    if (txtId.getText().trim().isEmpty()
        || txtName.getText().trim().isEmpty()
        || txtContact.getText().trim().isEmpty()
        || txtEmail.getText().trim().isEmpty()
        || txtBasic.getText().trim().isEmpty()) {

        txtArea.setText("Please fill all required fields.");
        return false;
    }

    try {

```

```

Integer.parseInt(txtId.getText().trim());
Double.parseDouble(txtBasic.getText().trim());
Double.parseDouble(txtAllowances.getText().trim());
Double.parseDouble(txtDeductions.getText().trim());

} catch (NumberFormatException ex) {

    txtArea.setText(
        "Employee ID and salary values must be valid numbers.");

    return false;
}
return true;
}

void addEmployee() {
    if (!validateData())
        return;

    String sql =
        "INSERT INTO employee "
        + "(employee_id,name,gender,dob,contact,email,address,"
        + "department,designation,joining_date,basic_salary,"
        + "allowances,deductions,status) "
        + "VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?)";

```

```
try {

    Connection con = DBconnection.getConnection();
    PreparedStatement ps = con.prepareStatement(sql);

    ps.setInt(1, Integer.parseInt(txtId.getText()));
    ps.setString(2, txtName.getText());
    ps.setString(3, gender.getSelectedItem());
    ps.setString(4, txtDob.getText());
    ps.setString(5, txtContact.getText());
    ps.setString(6, txtEmail.getText());
    ps.setString(7, txtAddress.getText());
    ps.setString(8, department.getSelectedItem());
    ps.setString(9, designation.getSelectedItem());
    ps.setString(10, txtJoiningDate.getText());
    ps.setDouble(11, Double.parseDouble(txtBasic.getText()));
    ps.setDouble(12, Double.parseDouble(txtAllowances.getText()));
    ps.setDouble(13, Double.parseDouble(txtDeductions.getText()));
    ps.setString(14, status.getSelectedItem());

    ps.executeUpdate();
    txtArea.setText("Employee added successfully.");
    ps.close();
    con.close();

} catch (SQLIntegrityConstraintViolationException ex) {
```

```

        txtArea.setText("Employee ID already exists.");
    } catch (Exception ex) {

        txtArea.setText("Database Error: " + ex.getMessage());
    }
}

void searchEmployee() {
    if (txtId.getText().trim().isEmpty()) {
        txtArea.setText("Enter Employee ID.");
        return;
    }

    String sql =
        "SELECT * FROM employee WHERE employee_id=?";
    try {
        Connection con = DBconnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql);
        ps.setInt(1, Integer.parseInt(txtId.getText()));
        ResultSet rs = ps.executeQuery();

        if (rs.next()) {

            txtName.setText(rs.getString("name"));
            gender.select(rs.getString("gender"));
            txtDob.setText(rs.getString("dob"));

```

```
txtContact.setText(rs.getString("contact"));
txtEmail.setText(rs.getString("email"));
txtAddress.setText(rs.getString("address"));
department.select(rs.getString("department"));
designation.select(rs.getString("designation"));
txtJoiningDate.setText(rs.getString("joining_date"));
txtBasic.setText(rs.getString("basic_salary"));
txtAllowances.setText(rs.getString("allowances"));
txtDeductions.setText(rs.getString("deductions"));
status.select(rs.getString("status"));

txtArea.setText("Employee found successfully.");

} else {

    txtArea.setText("Employee not found.");
}

rs.close();
ps.close();
con.close();

} catch (Exception ex) {

    txtArea.setText("Database Error: " + ex.getMessage());
}
```



```
}
```

```
void updateEmployee() {
```

```
    if (!validateData())
```

```
        return;
```

```
    String sql =
```

```
        "UPDATE employee SET name=?,gender=?,dob=?,contact=?,"
```

```
        + "email=?,address=?,department=?,designation=?,"
```

```
        + "joining_date=?,basic_salary=?,allowances=?,"
```

```
        + "deductions=?,status=? WHERE employee_id=?";
```

```
    try {
```

```
        Connection con = DBconnection.getConnection();
```

```
        PreparedStatement ps = con.prepareStatement(sql);
```

```
        ps.setString(1, txtName.getText());
```

```
        ps.setString(2, gender.getSelectedItem());
```

```
        ps.setString(3, txtDob.getText());
```

```
        ps.setString(4, txtContact.getText());
```

```
        ps.setString(5, txtEmail.getText());
```

```
        ps.setString(6, txtAddress.getText());
```

```
        ps.setString(7, department.getSelectedItem());
```

```
        ps.setString(8, designation.getSelectedItem());
```

```
ps.setString(9, txtJoiningDate.getText());  
ps.setDouble(10, Double.parseDouble(txtBasic.getText()));  
ps.setDouble(11, Double.parseDouble(txtAllowances.getText()));  
ps.setDouble(12, Double.parseDouble(txtDeductions.getText()));  
ps.setString(13, status.getSelectedItem());  
ps.setInt(14, Integer.parseInt(txtId.getText()));
```

```
int result = ps.executeUpdate();
```

```
if (result > 0) {
```

```
    txtArea.setText("Employee updated successfully.");
```

```
} else {
```

```
    txtArea.setText("Employee not found.");
```

```
}
```

```
ps.close();
```

```
con.close();
```

```
} catch (Exception ex) {
```

```
    txtArea.setText("Database Error: " + ex.getMessage());
```

```
}
```

```
}
```

```
void deleteEmployee() {

    if (txtId.getText().trim().isEmpty()) {

        txtArea.setText("Enter Employee ID.");
        return;
    }

    String sql =
        "DELETE FROM employee WHERE employee_id=?";

    try {

        Connection con = DBconnection.getConnection();

        PreparedStatement ps = con.prepareStatement(sql);

        ps.setInt(1, Integer.parseInt(txtId.getText()));

        int result = ps.executeUpdate();

        if (result > 0) {

            txtArea.setText("Employee deleted successfully.");
```

```

    } else {

        txtArea.setText("Employee not found.");
    }

    ps.close();
    con.close();

} catch (Exception ex) {

    txtArea.setText("Database Error: " + ex.getMessage());
}
}

void viewEmployees() {

    String sql =
        "SELECT employee_id,name,department,designation,"
        + "basic_salary,status FROM employee";

    try {

        Connection con = DBconnection.getConnection();

        Statement st = con.createStatement();

```

```
ResultSet rs = st.executeQuery(sql);
```

```
StringBuilder result = new StringBuilder();
```

```
result.append("EMPLOYEE DETAILS\n");
```

```
result.append("-----\n");
```

```
while (rs.next()) {
```

```
    result.append("ID : ")
```

```
        .append(rs.getInt("employee_id"))
```

```
        .append("\n");
```

```
    result.append("Name : ")
```

```
        .append(rs.getString("name"))
```

```
        .append("\n");
```

```
    result.append("Department : ")
```

```
        .append(rs.getString("department"))
```

```
        .append("\n");
```

```
    result.append("Designation : ")
```

```
        .append(rs.getString("designation"))
```

```
        .append("\n");
```

```
    result.append("Basic Salary : ")
```

```

        .append(rs.getDouble("basic_salary"))
        .append("\n");

    result.append("Status : ")
        .append(rs.getString("status"))
        .append("\n");

    result.append("-----\n");
}

txtArea.setText(result.toString());

rs.close();
st.close();
con.close();

} catch (Exception ex) {

    txtArea.setText("Database Error: " + ex.getMessage());
}
}

void calculatePayroll() {

    try {

```

```
double basic =
```

```
    Double.parseDouble(txtBasic.getText());
```

```
double allowance =
```

```
    Double.parseDouble(txtAllowances.getText());
```

```
double deduction =
```

```
    Double.parseDouble(txtDeductions.getText());
```

```
double grossSalary =
```

```
    basic + allowance;
```

```
double netSalary =
```

```
    grossSalary - deduction;
```

```
txtArea.setText(
```

```
    "===== PAYROLL DETAILS =====\n\n"
```

```
    + "Employee ID   : " + txtId.getText() + "\n"
```

```
    + "Employee Name : " + txtName.getText() + "\n\n"
```

```
    + "Basic Salary  : " + basic + "\n"
```

```
    + "Allowances    : " + allowance + "\n"
```

```
    + "Deductions     : " + deduction + "\n"
```

```
    + "-----\n"
```

```
    + "Gross Salary   : " + grossSalary + "\n"
```

```
    + "Net Salary    : " + netSalary
```

```
);
```

```
    } catch (NumberFormatException ex) {  
  
        txtArea.setText(  
            "Enter valid salary values.");  
    }  
}
```

```
void clearFields() {  
  
    txtId.setText("");  
    txtName.setText("");  
    txtDob.setText("");  
    txtContact.setText("");  
    txtEmail.setText("");  
    txtAddress.setText("");  
    txtJoiningDate.setText("");  
    txtBasic.setText("");  
    txtAllowances.setText("");  
    txtDeductions.setText("");  
  
    gender.select(0);  
    department.select(0);  
    designation.select(0);  
    status.select(0);  
}
```



```
txtArea.setText("Fields cleared.");  
}
```

```
public static void main(String[] args) {
```

```
    new EmployeeCRUD();  
}  
}
```